

Algorithmen und Datenstrukturen

Übung 2 – Grundlagen C und C++

Aufgabe 1: Allgemeine Grundlagen der Programmierung

a) Beantworten Sie die folgenden Grundlagenfragen:

- Was ist der Unterschied zwischen *lokalen Variablen* und *globalen Variablen*?
- Welchen Vorteil hat eine Liste gegenüber einem Array, welchen Nachteil?

Falls Sie zudem Erfahrungen in der objektorientierten Programmierung haben, beantworten Sie weiterhin die folgenden Fragen:

- Was ist der Unterschied zwischen einer *Funktion* und einer *Methode*?
Hinweis: Funktion und Methode werden trotz der semantischen Unterschiede in dieser Veranstaltung weitgehend synonym verwendet.
- Was ist der Unterschied zwischen *Instanzvariablen* und *Klassenvariablen*?
- Was für einen Unterschied macht es, ob ich (z. B. in Java) eine Methode mit `return x;` oder mit `throw x;` beende?

b) Stellen Sie sich vor, Sie sind Projektleiter bei einer Firma und haben im Rahmen der nachstehenden Software-Projekte eine Programmiersprache auszuwählen. Welche Wahl treffen Sie und warum?

- Im Top-Manager-Simulator *MoneyMatters* wurde der Mehrspielermodus aus Budget-Gründen gestrichen. Trotzdem soll zumindest eine Punktetabelle als Website über das Internet gepflegt werden, damit man mit Freunden um das beste, virtuelle Unternehmen wetteifern kann. Deshalb ist von Ihnen ein kleiner Dienst zu entwickeln, der übers Internet erreichbar ist und der vom Spiel Daten erhält. Weil das Spiel bislang kein gutes Medienecho erzeugt hat, ist von eher geringer Last auszugehen. Ihre Sorge ist mehr, dass sich kriminelle Subjekte an der Datenbank zu schaffen machen. Das Startup trägt mit seiner Besetzung aus Neulingen und Quereinsteigern nicht gerade dazu bei, die Sorge zu lindern.
- Die *Superfit*-Gesundheitskasse startet mit dem Projekt *GrollCollect* durch. Da mittlerweile erwiesen ist, dass gute Laune die Krankheitstage senkt, soll dieses Projekt auch finanzielle Anreize für die eigene Gesundheit schaffen. Über eine von Ihnen zu entwickelnde Desktop-Anwendung sollen Sachbearbeiter regelmäßig auf die Handycameras der Kassenkunden zugreifen, um deren Gefühlszustand zu erfassen und bei schlechter Laune Kassengebühren nachzufordern. Die Herausforderung liegt einerseits in der knappen Zeitplanung, andererseits in der Integration mit bestehenden Anwendungen der Gesundheitskasse. Zum Glück ist Ihr Budget nahezu unbeschränkt und das Programmiererteam arbeitet seit Jahren an ähnlichen Projekten.

- Der Autohersteller *JCM Electro* möchte zahlungskräftige Kunden mit einem neuen Feature locken: Ein Fuß-Jacuzzi im Wagenboden, das selbstlernende Massageprogramme für gestresste Fahrerfüße bietet. Aufgrund elektrischer Bedenken ist eine Anbindung des Jacuzzis an den Fahrzeugbus nicht möglich. Um die Marge nicht unnötig einzuschränken, zwingt die Firmenleitung Sie zur Realisierung aller Funktionen auf einem ATmega-Mikrocontroller mit 2 KB RAM und 20 MHz CPU-Takt. Die eigentliche Programmierung soll dann kostengünstig ins Ausland verlagert werden.
- Für das junge Unternehmen *Personal Perfection* arbeiten Sie an einer neuen Geschäftsidee: Eine ständig laufende App fürs Handy soll dazu mit Kamera und Mikrofon den gesamten Tag über am Alltag der Kunden teilhaben und deren Leben durch unterschwellige Interaktion (z. B. leise Tonausgaben, Anzeigen im Newsfeed, automatische Nachrichten an soziale Netze) in die optimale Richtung entwickeln. Zur Datenanalyse und zur Wahl der richtigen Steuerungsmöglichkeit kommt eine komplexe, selbsterstellte KI zum Einsatz, jedoch darf diese aus völlig ungerechtfertigten Datenschutzbedenken nur auf der lokalen Hardware im Mobiltelefon laufen.
- Der Industriekonzern *Flibbers and Snoggles* stellt fast jedes Produkt her, von Eisenwaren bis zu Medikamenten. Die Standorte sind über die Welt verteilt und arbeiten bisher weitgehend unabhängig. Im Sinne eines zentralen Controllings soll jetzt aber unter Ihrer Leitung eine universelle Datenbank aller Konzernprodukte erstellt werden, um einen strengen Sparplan umzusetzen. Module zum Datenimport sollen im Sinne der Sparmaßnahmen von den fachlich unversierten Standorten selbst geschrieben werden. Ihre Aufgabe ist es, eine passende Basisanwendung für dieses Unterfangen zu erstellen, die sowohl über den Desktop, als auch über Mobilgeräte einen Zugriff auf den Datenkatalog sowie das Abarbeiten der Importmodule erlaubt.

Hinweis: Es genügt, wenn Sie sich auf Sprachen bzw. Sprachtypen aus Ihrem Studium oder der Vorlesung (z. B. Assembler, Python, Java, C, C++) beschränken. Sie dürfen gerne andere Sprachen wählen, die Sie kennen, sofern die Sprachen nicht zu kurios sind.

Aufgabe 2: Die Compilerpipeline

- Beschreiben Sie in eigenen Worten den Weg von C++-Quelldateien zum ausführbaren Programm. Gehen Sie insbesondere auf die Bedeutung von Compiler und Linker für den Erstellungsprozess ein, sowie auf die erstellten Zwischen- und Ausgabedateien.
- Wenn zwei Übersetzungseinheiten Inhalte der jeweils anderen Einheit verwenden, liegt eine zyklische Abhängigkeit vor. Welches Problem verursacht eine solche Abhängigkeit im Rahmen der Compilerpipeline von C++? Wie ist das Problem zu lösen?

Aufgabe 3: Quellcodedateien und Include-Anweisungen

- a) Beschreiben Sie in eigenen Worten den Nutzen von Headerdateien. Welche Programmteile werden in Headerdateien ausgelagert, welche Programmteile können in Quellcodedateien abgelegt werden?
- b) Inclusion Guards benötigen dringend eindeutige Namen. Allerdings wird das weder geprüft, noch erzwungen. Was muss passieren, damit ein doppelt vergebener Name für eine Inclusion Guard ein sichtbares Problem verursacht? Wie äußert sich das Problem und warum ist es schwer zu diagnostizieren?
- c) In der Vorlesung wird eine feste Reihenfolge für Include-Anweisungen in C++-Programmen vorgeschlagen. Warum ergibt es Sinn, eigene Header erst nach den Headern von Standard- und Fremdbibliotheken einzubinden?

Kompilieren Sie dazu probierhalber folgendes Codefragment:

```
1 // Datei vector.hpp
2 #if !defined(INCLUDED__BLATT1__VECTOR_HPP)
3 #define INCLUDED__BLATT1__VECTOR_HPP
4     namespace std { class vector; }
5 #endif
6
6 // Datei main.cpp
7 #include "vector.hpp"
8 #include <vector>
9 int main() { }
```

- d) In der Vorlesung wird ein Spezialfall für Includes in C++-Quellcodedateien vorgeschlagen: Der zugehörige Header ist immer als erster einzubinden. Warum ergibt diese Forderung Sinn?

Nutzen Sie zur Argumentation folgendes Beispiel: Zuerst werden die zwei Dateien `output.hpp` und `output.cpp` von einem Softwareentwickler erstellt.

```
1 // Datei output.hpp
2 #if !defined(INCLUDED__BLATT1__OUTPUT_HPP)
3 #define INCLUDED__BLATT1__OUTPUT_HPP
4     void output_text(const std::string& text);
5 #endif
6
6 // Datei output.cpp
7 #include <string>
8 #include <iostream>
9
9 #include "output.hpp"
10
10 void output_text(const std::string& text)
11     { std::cout << text << std::endl; }
```

Ein anderer Entwickler schreibt deutlich später die folgende Datei:

```
12 // Datei main.cpp
13 #include "output.hpp"
14 int main() { output_text("Hello"); }
```

- e) Inclusion Guards können weggelassen werden. Das geschieht in vielen Fällen unabsichtlich und verursacht über doppelte Definitionen wenig aussagekräftige Compilefehler. Theoretisch könnte der Compiler das mehrfache Einbinden einer Datei direkt erkennen und ignorieren oder mit einer sprechenden Fehlermeldung verbieten. Warum wird das nicht so gemacht? Geben Sie ein Anwendungsbeispiel in Form von konkretem Code, in dem die mehrfache textuelle Einbindung einer Datei Sinn macht. Bedenken Sie dabei, dass der Präprozessor rekursiv auf allen eingebundenen Dateien läuft und dass es sich bei der eingebundenen Datei nicht zwangsläufig um einen vollständigen Header handeln muss.

Hinweis: Das mehrfache Einbinden einer Datei macht natürlich nur Sinn, wenn sich der Kontext der Einbindung zwischen den Einbindungsstellen ändert. Siehe dazu auch die nachfolgende Aufgabe.

Aufgabe 4: Der Präprozessor

- a) Die Hauptaufgabe des Präprozessors ist es nicht, Inclusion Guards zu realisieren. Stattdessen kann der Präprozessor vor dem Kompilieren nahezu beliebige textuelle Ersetzungen vornehmen.

Vervollständigen Sie folgendes Programmfragment an und nur an der markierten Stelle, dass das Programm den Text "Preprocessed" auf der Konsole ausgibt:

```
1 #include <iostream>
2 #define PRINT_PREPROCESSED /* ??? */
3 int main( PRINT_PREPROCESSED )
```

- b) Welche Ausgabe verursacht das folgende Programmfragment? Warum?

```
1 #include <iostream>
2 #define MULT(a, b) a * b

3 int main()
4 {
5     std::cout << MULT(2, 3) << std::endl;
6     std::cout << MULT(2.f, 3.f) << std::endl;
7     std::cout << MULT(1 + 4, 5 + 3) << std::endl;
8     std::cout << MULT((1 + 4), (5 + 3)) << std::endl;
9     std::cout << MULT("1" << 4, 5 << "2") << std::endl;
10    std::cout << MULT(MULT(2, 3), 4) << std::endl;
11 }
```

- c) Die Nutzung des Präprozessors zur Erzeugung von Quellcode ist auch als Makroprogrammierung bekannt. Kritisieren Sie im Hinblick auf die Erkenntnisse aus den vorangehenden Teilaufgaben die Makroprogrammierung in C++.

Aufgabe 5: Projekt mit mehreren Abhängigkeiten

a) Legen Sie die folgenden Übersetzungseinheiten als Paar aus .hpp und .cpp nach üblicher Konvention mit gegebenem Inhalt in einem gemeinsamen Projekt an:

- ein Modul battery mit einer Klasse battery, die eine einzige Member-Methode `void power_on()` hat. Die Methode `power_on()` soll bei Aufruf (also in der Implementierung der Methode) den Text "starting up" mit `iostream` auf der Kommandozeile ausgeben.
- ein Modul robot mit einer Klasse robot, die eine einzige Membervariable vom Typ `battery` hat. Der Default-Konstruktor von `robot` soll `power_on()` auf dieser Membervariable aufrufen und dann "Robot ready" ausgeben.
- ein Modul car mit einer Klasse car, die eine einzige Membervariable vom Typ `battery` hat. Der Default-Konstruktor von `car` soll wieder `power_on()` auf der Membervariable aufrufen und dann "car ready" ausgeben.
- ein Modul main mit einer freien Main-Methode (ohne Klasse), die erst einen `robot`, anschließend ein `car` auf dem Stack erzeugt.

Verwenden Sie für alle Variablen nur den Stack als Speicherort. Achten Sie ferner darauf, die Header korrekt umzusetzen. Dazu gehören korrekte Inclusion Guards, eine korrekte Aufteilung von Inhalten zwischen Header und Quellcode-Datei, sowie zuletzt eine korrekte Sortierung der `#include`-Anweisungen nach Vorlesung.

b) Passen Sie die Lösung der vorigen Aufgabe an: Behalten Sie die Klassenaufteilung auf die .cpp-Dateien bei, aber verzichten Sie auf die Verwendung eigener Header. Systemheader dürfen Sie nutzen – aber um Himmels willen nicht bearbeiten! Entgegen aller Konventionen den Headern andere Dateiendungen zu geben (z. B. die Header in .cpp umbenennen und `#inc1uden`) ist ebenfalls keine Lösung!

Aufgabe 6: Schöner Code

a) Geben Sie für die folgenden (unabhängigen) Codebeispiele jeweils eine möglichst einfache Darstellung mit nur einer einzigen Anweisung an.

```
// Beispiel 1
bool robot_ready()
{
    bool result = battery_loaded() && os_started();
    return result;
}

// Beispiel 2
bool is_obstacle_close()
{
    if (obstacle_detected() && distance < 3.0) return true;
    else return false;
}
```

```
// Beispiel 3
string rotate_direction("none");

if (obstacle_pos < robot_pos) rotate_direction = "left";
else if (obstacle_pos >= robot_pos) rotate_direction = "right";
```

Wo liegt ein besonderer Vorteil der kompakten Darstellung bei Beispiel 3?

- b) Vereinfachen Sie alle ifs in initialize() soweit möglich, ohne die Semantik der Funktion zu verändern. Verwenden Sie keine Exceptions, keine Zusatzfunktionen und keine Zusatzvariablen für diese Vereinfachung!

```
bool initialize()
{
    if (power_available())
    {
        power_on();

        if (terminal_connected())
        {
            show_logon_prompt();

            if (check_user_credentials())
            {
                show_welcome_user_message();
                return true;
            }
        }
    }
    return false;
}
```

Aufgabe 7: Grundlagen Stack, Heap, Zeiger

- a) Gegeben sei das nachstehende Codefragment:

```
#include <string>
#include <iostream>

struct point3f { float x; float y; float z; };

class robot
{
public:
    float speed;
    point3f position;
};

int main(int argc, char ** argv)
{
    float value = 5.f;
    float* value_ptr = &value;

    std::string s;
    std::string* s_ptr = &s;

    robot r;
    robot* r_ptr = &r;
}
```

Geben Sie für jede nachfolgende Variable an, ob diese Variable einen eingebauten oder einen zusammengesetzten ("Klassen-")Typen hat.

- die Variablen `value` und `value_ptr`
- die Variablen `s` und `s_ptr`
- die Variablen `r` und `r_ptr`
- die Variablen `r.speed`, `r.position` und `r.position.x`

Hinweis: Nach Standard ist der einzige Unterschied zwischen `struct` und `class`, dass eine `struct` als Voreinstellung `public:` verwendet, während die `class` als Voreinstellung `private:` nutzt. In der Praxis steht die Konvention, `structs` eher für reine Datencontainer ohne Methoden zu verwenden, während `class` für Klassen mit komplexem Verhalten und vielen Methoden steht.

Hinweis: Wie viele Member hat ein Zeiger? Rechtsklicken Sie zudem in Ihrer IDE auf den Bezeichner `string` und wählen Sie im Menü `Go-To-Definition` aus. Hangeln Sie sich von dort soweit vor, wie möglich, um eine Erkenntnis zu gewinnen.

b) Gegeben sei das folgende Codebeispiel:

```
struct sensor { float scanning_range; };

struct robot
{
    robot() : s(new sensor()) { }

    sensor* s;
    float speed;
    robot* partner_ptr;
};

void main()
{
    robot ra;
    robot *rb = new robot();
    robot *rc = &ra;

    rb->partner_ptr = &ra;

    /* !!! */

    /* ... Speicherfreigabe ... */
}
```

Angenommen, das Codebeispiel ist bis zur `!!!`-markierten Zeile gelaufen. Liegen die mittels der folgenden Ausdrücke lokalisierten Variablen in diesem Fall jeweils auf dem Stack oder auf dem Heap?

- Fall 1: `ra`
- Fall 2: `ra.speed`
- Fall 3: `ra.s`
- Fall 4: `ra.s->scanning_range`
- Fall 5: `rb`

- Fall 6: *rb
- Fall 7: rb->speed
- Fall 8: rb->s->scanning_range
- Fall 9: rc
- Fall 10: *rc
- Fall 11: rb->partner_ptr
- Fall 12: *(rb->partner_ptr)

c) Geben Sie die Gültigkeitsbereiche aller Stack- und Heap-Variablen in der Methode run() an. Betrachten Sie, wann Speicher allokiert bzw. freigegeben wird. Tragen Sie die jeweiligen Zeilennummern in die Tabelle ein. Falls keine zutreffende Zeile existiert, tragen Sie ein Minus ein. Welche drei, konzeptuell leicht unterschiedlichen Probleme treten in dem Programm auf?

Hinweis: w.exit() ist eventuell schon zu Programmstart true, zum Beispiel weil der Benutzer während dem Programmstart Alt+F4 gedrückt hat.

```

1   void run()
2   {
3       int* a = new int(5);
4       int* b;
5       window w(1920, 1080);

6       while (!w.exit())
7       {
8           b = new int(1);

9           draw_something(w, b);
10        }

11        delete b;
12    }

```

Variablen	Speicher auf dem Stack		Speicher auf dem Heap	
	allokiert	freigegeben	allokiert	freigegeben
a bzw. *a				
b bzw. *b				
w				

d) Gegeben sei das folgende Codebeispiel:

```
1 struct pumpkin
2 {
3     pumpkin()
4         { candles = 15; teeth = new int(32); eyes = 21; }
5
6     int candles;
7     int *teeth;
8     int eyes;
9 };
10
20 int main(int argc, char** argv) { pumpkin p; }
```

Welches Problem tritt auf? Wie kann man das Problem durch eine Änderung **nur** an der `main()` lösen?

Aufgabe 8: Fehler mit Zeigern

a) Schreiben Sie vier `main()`-Methoden, eine für jeden der folgenden Fälle:

- Schreibe den `int`-Wert 1337 an die Adresse 0 bis 3
- Schreibe den `int`-Wert 1337 an die feste Adresse 19522430 bis ...3
- Schreibe den `int`-Wert 1337 auf eine bereits deallokierte `int`-Stackvariable
- Schreibe den `int`-Wert 1337 auf eine bereits deallokierte `int`-Heapvariable

Geben Sie jeweils nach dem Schreibzugriff eine Nachricht auf der Konsole aus.

Was passiert im Debug-Build beim Ausführen des Programms in jedem der Fälle? Achten Sie gegebenenfalls darauf, in welcher Zeile die Ausführung vom Debugger unterbrochen wird. Ist das die Zeile mit dem kritischen Schreibzugriff?

Stellen Sie den Build-Modus anschließend auf Release um. Wie unterscheidet sich das Verhalten der vier Fälle in diesem Modus vom Debug-Build?

Geben Sie den Code (zusätzlich zur Beantwortung der Fragen) ab!

Hinweis: Unter Visual Studio können Sie auf den Projekteinstellungen (Rechtsklick auf das Projekt → Eigenschaften → C / C++ → Code Generation) einige optionale, sehr sinnvolle, aber hier eventuell störende Sicherheitsfeatures abstellen.

b) Prüfen Sie die nachfolgenden Codebeispiele unabhängig voneinander auf Fehler. So vorhanden, geben Sie den ersten Fehler an, der im Rahmen vom Bauen und Testen des Codestücks auftritt. Erklären Sie, was den Fehler verursacht – diese Erklärung soll zeigen, dass Sie den Fehler verstanden haben. **Das Abtippen einer Compiler-Fehlermeldung genügt nicht!** Geben Sie ferner an, in welcher Zeile der Fehler auftritt und ob es sich um einen Compilezeit- oder Laufzeitfehler handelt. Bei Codestücken ohne Fehler geben Sie stattdessen die Ausgabe des Codes an.

Hinweis: Gegebenenfalls vorhandene Fehler hängen immer unmittelbar mit den dargestellten Codezeilen zusammen. Unnötige Details (z. B. Header oder Rümpfe von Funktionen) wurden aus Gründen der Kompaktheit bei den Code-Fragmenten weggelassen.

Hinweis: Unvollständige Lösungen (z. B. ohne Begründung bei einem Fehler) geben keine Punkte auf das entsprechende Beispiel. Raten hilft hier nicht!

Hinweis: Compilerwarnungen sind keine Fehler, weisen aber oft auf Fehler hin!

```
1 // Beispiel 1
2 int i = 5;
3 int *p0, *p1;
4 p1 = &i;
5 p0 = &p1;
6 std::cout << "Ergebnis: " << *p0;
```

```
1 // Beispiel 2
2 int *p = nullptr;
3 int **pp = &p;
4 int i = 5;
5 *pp = &i;
6 std::cout << "Ergebnis: " << *p;
```

```
1 // Beispiel 3
2 int *p = new int();
3 delete p;
4 if (p == nullptr) std::cout << "True in z4";
5 p = nullptr;
6 std::cout << p;
7 std::cout << *p;
```

```
1 // Beispiel 4
2 float *f = new float;
3 delete &f;
4 f = nullptr;
5 std::cout << "Ergebnis: " << f;
```

c) Recherchieren Sie, was man unter (*strict*) *pointer aliasing* versteht. Erklären Sie damit, welches Fehlverhalten das folgende Codebeispiel zeigen kann:

```
1 int main()
2 {
3     float f = 5.0f;
4     float *ptr_f = &f;
5     int *ptr_i = (int*)&f;
6
7     std::cout << "f hat Hex-wert: " << std::hex << *ptr_i;
8     *ptr_f = 10.0f;
9     std::cout << "f hat Hex-wert: " << std::hex << *ptr_i;
10 }
```

Hinweis: Falls das Codebeispiel bei Ihnen kein auffälliges Verhalten produziert, dann ist Ihr Compiler beim Erzeugen von Maschinencode entweder zu aggressiv oder zu vorsichtig (sic!). Sie können versuchen, die Code-Optimierung an- bzw. abzuschalten (z. B. durch einen Wechsel zwischen Debug- und Release-Build), um eventuell das gesuchte Fehlverhalten zu erzwingen.